

Ubuntu eBPF Benign Telemetry Collector Instructions

Use this guide for the team member collecting benign telemetry on Ubuntu. This version uses eBPF/BCC and keeps the same column structure used in the final dataset.

Step 1 - Create folders

```
mkdir -p ~/FYP_Keylogger/scripts
mkdir -p ~/FYP_Keylogger/dataset/benign
cd ~/FYP_Keylogger/scripts
```

Step 2 - Install requirements

```
sudo apt update
sudo apt install python3-pip bpfcc-tools python3-bpfcc linux-headers-$(uname -r) -y
pip3 install pandas psutil --break-system-packages
```

If Ubuntu does not require `--break-system-packages`, use `pip3 install pandas psutil` instead.

Step 3 - Create the collector file

```
nano benign_ubuntu_ebpf_collector.py
```

Paste the collector code from the next section into this file.

Step 4 - Save the file

```
CTRL + O
ENTER
CTRL + X
```

Step 5 - Run the collector

```
sudo python3 benign_ubuntu_ebpf_collector.py
```

Step 6 - Check if rows are increasing

```
watch -n 1 "wc -l ~/FYP_Keylogger/dataset/benign/benign_ubuntu_ebpf.csv"
```

Collector Code

Copy this code into the file name mentioned in the instructions. Do not change the column names unless the whole team changes them together.

```
#!/usr/bin/env python3
import os
import sys
import time
from datetime import datetime
import pandas as pd
import psutil
from bcc import BPF

if os.geteuid() != 0:
    print("Run with sudo: sudo python3 benign_ubuntu_ebpf_collector.py")
    sys.exit(1)

print("Collecting Ubuntu BENIGN eBPF telemetry... Press CTRL+C to stop.")

os.makedirs("../dataset/benign", exist_ok=True)
csv_file = "../dataset/benign/benign_ubuntu_ebpf.csv"

bpf_source = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

BPF_HASH(sys_read_counter, u32, u64);
BPF_HASH(sys_write_counter, u32, u64);

int kprobe__sys_read(struct pt_regs *ctx) {
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    u64 init_val = 1, *val;
    val = sys_read_counter.lookup(&pid);
    if (val) {
        *val += 1;
    } else {
        sys_read_counter.update(&pid, &init_val);
    }
    return 0;
}

int kprobe__sys_write(struct pt_regs *ctx) {
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    u64 init_val = 1, *val;
    val = sys_write_counter.lookup(&pid);
    if (val) {
        *val += 1;
    } else {
        sys_write_counter.update(&pid, &init_val);
    }
    return 0;
}
"""

bpf_module = BPF(text=bpf_source)

def get_hardware_keyboard_interrupts():
    try:
        with open("/proc/interrupts", "r") as f:
            for line in f:
                line_lower = line.lower()
                if any(k in line_lower for k in ["i8042", "atkbd", "keyboard", "hid", "isa0060"]):
                    return sum([int(x) for x in line.split()[1:] if x.isdigit()])
    except Exception:
        pass
    return 0

prev_interrupts = get_hardware_keyboard_interrupts()
prev_ctx_switches = psutil.cpu_stats().ctx_switches

try:
    while True:
        read_map = bpf_module.get_table("sys_read_counter")
        write_map = bpf_module.get_table("sys_write_counter")

        total_kernel_reads = sum([val.value for key, val in read_map.items()])
        total_kernel_writes = sum([val.value for key, val in write_map.items()])

        read_map.clear()
        write_map.clear()

        actual_interrupts = get_hardware_keyboard_interrupts()
        keyboard_events_delta = max(0, actual_interrupts - prev_interrupts)

        actual_ctx_switches = psutil.cpu_stats().ctx_switches
        ctx_switches_delta = max(0, actual_ctx_switches - prev_ctx_switches)
```

```

prev_interrupts = actual_interrupts
prev_ctx_switches = actual_ctx_switches

cpu = psutil.cpu_percent(interval=0.1)
memory = psutil.virtual_memory().percent
swap_memory = psutil.swap_memory().percent
disk_usage = psutil.disk_usage('/').percent
system_uptime = round((time.time() - psutil.boot_time()) / 60, 2)
process_count = len(psutil.pids())
cpu_threads = psutil.cpu_count()

keyboard_events = process_count
total_open_files = process_count

all_processes = list(psutil.process_iter([
    'pid', 'name', 'cpu_percent', 'memory_percent', 'terminal', 'status'
]))

active_processes = 0
background_processes = 0
high_cpu_processes = 0
high_memory_processes = 0
python_processes = 0
shell_processes = 0
suspicious_process_names = 0
zombie_processes = 0
total_threads = 0

suspicious_names = ["keylogger", "logger", "hook", "monitor", "spy", "capture", "record"]

for proc in all_processes:
    try:
        active_processes += 1
        name = proc.info['name'] or ""

        if proc.info['terminal'] is None:
            background_processes += 1
        if proc.info['cpu_percent'] and proc.info['cpu_percent'] > 10:
            high_cpu_processes += 1
        if proc.info['memory_percent'] and proc.info['memory_percent'] > 5:
            high_memory_processes += 1
        if "python" in name.lower():
            python_processes += 1
        if name.lower() in ["bash", "sh", "zsh"]:
            shell_processes += 1
        if any(keyword in name.lower() for keyword in suspicious_names):
            suspicious_process_names += 1
        if proc.info['status'] == psutil.STATUS_ZOMBIE:
            zombie_processes += 1

        total_threads += proc.num_threads()

    except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
        pass

network_connections = len(psutil.net_connections())
total_connections = network_connections
users_logged_in = len(os.popen("who").readlines())
timestamp_now = datetime.now()

keyboard_to_process_ratio = round(keyboard_events / max(process_count, 1), 4)
cpu_to_keyboard_ratio = round(cpu / max(keyboard_events, 1), 4)
thread_to_process_ratio = round(total_threads / max(process_count, 1), 4)

row = {
    "timestamp": timestamp_now,
    "hour": timestamp_now.hour,
    "minute": timestamp_now.minute,
    "second": timestamp_now.second,
    "cpu_usage": cpu,
    "memory_usage": memory,
    "swap_memory": swap_memory,
    "disk_usage_percent": disk_usage,
    "system_uptime_minutes": system_uptime,
    "process_count": process_count,
    "active_processes": active_processes,
    "background_processes": background_processes,
    "high_cpu_processes": high_cpu_processes,
    "high_memory_processes": high_memory_processes,
    "python_processes": python_processes,
    "shell_processes": shell_processes,
    "suspicious_process_names": suspicious_process_names,
    "zombie_processes": zombie_processes,
    "cpu_threads": cpu_threads,
    "total_threads": total_threads,
    "thread_to_process_ratio": thread_to_process_ratio,
    "keyboard_events": keyboard_events,
    "keyboard_to_process_ratio": keyboard_to_process_ratio,
    "total_open_files": total_open_files,

```

```

        "network_connections": network_connections,
        "total_connections": total_connections,
        "users_logged_in": users_logged_in,
        "cpu_to_keyboard_ratio": cpu_to_keyboard_ratio,
        "keyboard_hardware_interrupts": keyboard_events_delta,
        "kernel_context_switches_delta": ctx_switches_delta,
        "kernel_sys_read_delta": total_kernel_reads,
        "kernel_sys_write_delta": total_kernel_writes,
        "label": "benign"
    }

    df = pd.DataFrame([row])

    if os.path.exists(csv_file):
        df.to_csv(csv_file, mode='a', header=False, index=False)
    else:
        df.to_csv(csv_file, mode='w', header=True, index=False)

    print(f"[BENIGN UBUNTU] Read: {total_kernel_reads} | Write: {total_kernel_writes} | Label: benign")
    time.sleep(0.5)

except KeyboardInterrupt:
    print("\nUbuntu benign telemetry collection stopped.")

```

Collection Guidelines

While collecting benign data, do normal activities only: browsing, typing, opening terminals, file manager usage, watching videos, writing text, and idle periods.

Do not run keylogger scripts during benign collection.

Target 5,000-10,000 rows. Minimum acceptable collection is around 2,000 rows.

Output file: ~/FYP_Keylogger/dataset/benign/benign_ubuntu_ebpf.csv